E03 2D Sprite Shaders (optional)

CRE132 worksheet

What you'll make

Three sprite effects made with **shaders**: a glowing outline that appears round a slime when you walk up to it, a slime that pulses green as if poisoned, and a slime that burns away in a noisy pattern with a glowing orange edge when it dies.

A shader is a small program that decides the colour of every pixel of a sprite. You'll drive two ready-made shaders from scripts, then build the dissolve shader yourself in **Shader Graph**, Unity's node editor for shaders, so you don't need to write shader code.

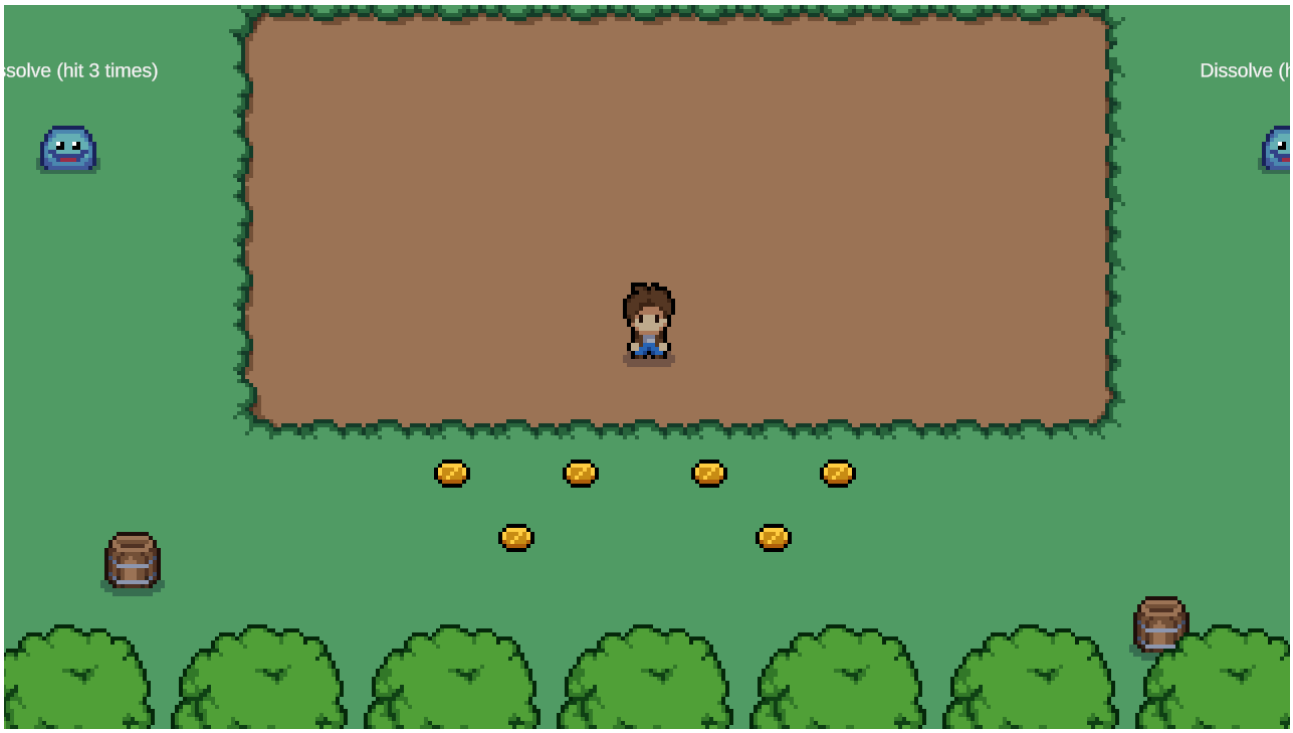


Figure 1: The finished scene: slimes labelled Outline, Tint and Dissolve, with an outline showing round the nearest one.

Before you start

1. Unzip `CRE132_E03_Shaders2D_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_E03_Shaders2D_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Effects/E03_Shaders2D`.
3. Press **Play**. Walk with WASD or the arrows and hit with **J** or the left mouse button. Labels float over the slimes: two say **Outline (walk up)**, one says **Tint** and two say **Dissolve (hit 3 times)**. The Outline slimes have an outline all the time, the Tint slime looks normal, and the Dissolve slimes just vanish after three hits.

The **Sprite Outline**, **Sprite Flash** and **Sprite Dissolve** materials are provided in `Assets/_game/Materials/Effects`, and the Outline and Tint slimes already use the first two. You'll build your own dissolve in Shader Graph; the provided one is there to compare against. Their shaders are in `Assets/`

`_game/Shaders`, written as short shader files. You don't need to read them, but they're there with comments if you're curious.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. A material is a shader plus settings

Expand a **Slime Dummy** labelled **Outline** and select its **Sprite** child. In the **Sprite Renderer**, **Material** is **Sprite Outline**. Double-click it: the Inspector shows its shader (**CRE132/Sprite Outline**) and its settings, **Outline Color** and **Outline Thickness**.

Press **Play**, then change **Outline Thickness** between 0 and 1 and watch the slime. Do the same with **Flash Amount** on the **Sprite Flash** material (used by the **Tint** slime). Stop Play, and set them back if they stayed changed (Outline Thickness 1, Flash Amount 0).

- **The shader is the program**; the **material** is one set of values for it. Many sprites can share one material.
- **Flash** blends each pixel towards **Flash Color**: 0 is the normal sprite, 1 a solid silhouette. E04 uses it for the white hit flash.
- **Outline** colours any see-through pixel that sits next to a solid one, so it needs empty space round the sprite. That's why the slime's sprite sheet has **Mesh Type: Full Rect**.

The scripts below change these values while the game runs.

2. A poisoned glow: PulseTint

Make a folder `Assets/_game/Scripts/E03_Shaders2D` and create `PulseTint` in it:

```
// Pulses a colour over the sprite (the CRE132/Sprite Flash shader with a colour), for
// example a poisoned slime glowing green. Mathf.Sin gives a smooth wave between -1 and 1.
using UnityEngine;

public class PulseTint : MonoBehaviour
{
    [SerializeField] private Color tint = new Color(0.4f, 1f, 0.3f);
    [SerializeField] private float strength = 0.6f;
    [SerializeField] private float pulsesPerSecond = 1.5f;

    private SpriteRenderer spriteRenderer;

    private void Awake()
    {
        spriteRenderer = GetComponentInChildren<SpriteRenderer>();
        spriteRenderer.material.SetColor("_FlashColor", tint);
    }

    private void Update()
    {
        // Sin goes from -1 to 1; turn it into 0 to 1, then scale it by the strength.
        float wave = (Mathf.Sin(Time.time * pulsesPerSecond * 2f * Mathf.PI) + 1f) / 2f;
        spriteRenderer.material.SetFloat("_FlashAmount", wave * strength);
    }
}
```

- **`SetColor` and `SetFloat`** change a material setting from code. The names (`_FlashColor`, `_FlashAmount`) are the shader's own names for the settings, not the labels you see in the Inspector.

- **`Mathf.Sin`** rises and falls smoothly between −1 and 1. Adding 1 and halving gives 0 to 1; times 0.6 means it never goes fully green.
- **`.material`** gives this renderer its own copy of the material, so only this slime changes. `.sharedMaterial` would change every sprite using Sprite Flash.

Add it to the **Slime Dummy** labelled **Tint** (the parent, not the Sprite child). Leave **Tint** light green, **Strength** 0.6 and **Pulses Per Second** 1.5.

Test it: press Play. The slime glows green and fades, one and a half times a second.

3. Outline when near: OutlineWhenNear

Create `OutlineWhenNear` in the same folder:

```
// Shows an outline round the sprite (the CRE132/Sprite Outline shader) while the player
// is close, a common way to say "you can interact with this".
using UnityEngine;

public class OutlineWhenNear : MonoBehaviour
{
    [SerializeField] private float range = 2f;

    private SpriteRenderer spriteRenderer;
    private Transform player;
    private bool isOutlined = true;

    private void Awake()
    {
        spriteRenderer = GetComponentInChildren<SpriteRenderer>();
        player = GameObject.FindWithTag("Player").transform;
        SetOutline(false);
    }

    private void Update()
    {
        bool playerIsNear = player != null &&
            Vector2.Distance(transform.position, player.position) <= range;
        if (playerIsNear != isOutlined)
        {
            SetOutline(playerIsNear);
        }
    }

    private void SetOutline(bool show)
    {
        isOutlined = show;
        float thickness = 0f;
        if (show)
        {
            thickness = 1f;
        }
        spriteRenderer.material.SetFloat("_OutlineThickness", thickness);
    }
}
```

- **`FindWithTag("Player")`** finds the player once, in `Awake`. The Player object has the **Player** tag.
- **`Vector2.Distance`** is how far apart the slime and player are. Within `range` (2 units) the outline shows.
- **Only change it when it needs changing:** `isOutlined` remembers the current state, so `SetOutline` runs once when you arrive and once when you leave, not every frame.

Add it to both **Slime Dummies** labelled **Outline**. Leave **Range** at 2.

Test it: walk up to one. A yellow outline appears; walk away and it goes.

4. Build a dissolve shader in Shader Graph

The dissolve needs a noise texture: a square of random greys. It's provided: `Assets/_game/Shaders/Textures/DissolveNoise`.

How it works: every pixel looks up a grey value (0 to 1) in the noise. Pixels whose noise is below **Dissolve Amount** disappear; pixels just above it glow orange. Raising the amount from 0 to 1 eats the sprite away, like burning paper.

In `Assets/_game/Shaders`, right-click and choose **Create > Shader Graph > URP > Sprite Unlit Shader Graph**. Name it `MySpriteDissolve` and double-click it to open the graph.

Properties. In the **Blackboard** (top left), click **+** to add each of these. Select a property to see its **Node Settings** in the Graph Inspector, and set the **Reference** exactly, underscore included: the scripts use these names.

Name	Type	Reference	Default
MainTex	Texture2D	`_MainTex`	(empty)
Noise	Texture2D	`_NoiseTex`	DissolveNoise
Noise Scale	Float	`_NoiseScale`	4
Dissolve Amount	Float, Mode **Slider** 0 to 1	`_DissolveAmount`	0
Edge Color	Color	`_EdgeColor`	orange, hex FF991A
Edge Width	Float, Mode **Slider** 0 to 0.3	`_EdgeWidth`	0.08

`_MainTex` is special: a Sprite Renderer puts the sprite's picture there.

Nodes. Drag each property onto the graph to use it. Right-click empty space and choose **Create Node** to search for the others. Then connect:

1. **Sample Texture 2D** with **MainTex** in its **Texture** input. This is the sprite.
2. A **UV** node into a **Multiply** with **Noise Scale**, into the **UV** input of a second **Sample Texture 2D** whose **Texture** is **Noise**. Its **R** output is the noise value. (Noise Scale 4 repeats the noise four times across the sprite, so the pattern is finer.)
3. **Step** with **Edge** = **Dissolve Amount** and **In** = noise **R**. Step gives 1 when In is at least Edge, otherwise 0: 1 means "keep this pixel".
4. **Multiply** that by the sprite's **A** (alpha) and connect it to **Alpha** in the **Fragment** block.
5. **Add Dissolve Amount** and **Edge Width**. Then a second **Step** with **Edge** = noise **R** and **In** = the Add result: 1 for pixels just above the cut-off. This is the edge.
6. A third **Step** with **Edge** 0.001 and **In** = **Dissolve Amount**, **Multiply**'d with the edge from step 5. Without it, a few pixels would glow even before the slime starts dissolving.
7. **Lerp** with **A** = the sprite's **RGBA**, **B** = **Edge Color**, **T** = the edge from step 6. Connect it to **Base Color**.

Click **Save Asset** (top left of the graph window).

Make the material: right-click `MySpriteDissolve` and choose **Create > Material**. Name it `My Sprite Dissolve` and move it to `Materials/Effects`. Check its **Noise** slot shows **DissolveNoise**. Drag **Dissolve Amount** in its Inspector and watch the preview square eat away. Select the provided **Sprite Dissolve** material and do the same: yours should behave just like it.

5. Dissolving on death: DissolveAway and DissolveOnDeath

Health destroys the slime straight after **On Died**, so there's no time to animate it. Instead, the slime leaves a *copy* of its sprite behind, and the copy dissolves. Create `DissolveAway` in the `E03_Shaders2D` folder:

```
// Raises the dissolve shader's Dissolve Amount from 0 to 1, then removes the object.
// DissolveOnDeath adds this to the copy it leaves behind.
using System.Collections;
using UnityEngine;

public class DissolveAway : MonoBehaviour
{
    public void Begin(float duration)
    {
        StartCoroutine(DissolveRoutine(duration));
    }

    private IEnumerator DissolveRoutine(float duration)
    {
        Material material = GetComponent<SpriteRenderer>().material;
        float time = 0f;
        while (time < duration)
        {
            time = time + Time.deltaTime;
            material.SetFloat("_DissolveAmount", time / duration);
            yield return null;
        }
        Destroy(gameObject);
    }
}
```

Then `DissolveOnDeath`, in the same folder:

```
// When this character dies, a copy of its sprite stays behind and dissolves away (the
// CRE132/Sprite Dissolve shader), instead of the character just vanishing. Health's
// On Died event calls Dissolve.
// The copy is a new object, because Health destroys the character straight after On Died.
using UnityEngine;

public class DissolveOnDeath : MonoBehaviour
{
    [SerializeField] private Material dissolveMaterial;
    [SerializeField] private float duration = 1f;

    public void Dissolve()
    {
        SpriteRenderer original = GetComponentInChildren<SpriteRenderer>();

        GameObject copy = new GameObject(gameObject.name + " (Dissolving)");
        copy.transform.position = original.transform.position;
        SpriteRenderer copyRenderer = copy.AddComponent<SpriteRenderer>();
        copyRenderer.sprite = original.sprite;
        copyRenderer.flipX = original.flipX;
        copyRenderer.sortingLayerID = original.sortingLayerID;
        copyRenderer.material = dissolveMaterial;

        // The copy runs the animation itself, because this object is about to be
        // destroyed.
```

```
        DissolveAway dissolve = copy.AddComponent<DissolveAway>();
        dissolve.Begin(duration);
    }
}
```

- **`new GameObject(...)`** makes an empty object in code, and **`AddComponent`** gives it a Sprite Renderer, just like Add Component in the Inspector. It copies the slime's current picture, facing and sorting layer.
- **`time / duration`** goes from 0 to 1 over one second, so **Dissolve Amount** rises and the sprite eats away. Then the copy destroys itself.
- **The copy runs its own coroutine.** A coroutine on the slime would stop when Health destroys it.

Add it: select both **Slime Dummies** labelled **Dissolve** and add **DissolveOnDeath**. Drag your **My Sprite Dissolve** material into **Dissolve Material** (if your graph won't work, use the provided **Sprite Dissolve** for now and come back to it); leave **Duration** at 1. Then, on each one's **Health**, find **On Died ()**. Click **+** to add an entry, drag in that **Slime Dummy** and choose **DissolveOnDeath > Dissolve ()**.

Test it: hit a Dissolve slime three times. It burns away over a second with an orange edge.

Check your work

- ☐ The Tint slime pulses green all the time.
- ☐ Walking up to an Outline slime shows a yellow outline; walking away hides it. Only that slime changes.
- ☐ A Dissolve slime burns away in a noisy pattern with an orange edge, and the "(Dissolving)" copy disappears from the Hierarchy afterwards.
- ☐ Compare with `CRE132_E03_Shaders2D_Finished` if something behaves differently.

Key points

- **A shader decides the colour of every pixel.** A tiny one, like the flash (one blend), can change how an object feels.
- **Drive shader properties from scripts** with `SetFloat` and `SetColor` for gameplay effects: hover outlines, damage flashes, deaths.
- **`.material` vs `.sharedMaterial`:** `.material` gives one renderer its own copy; `.sharedMaterial` changes every object using it.
- **These shaders are unlit:** 2D lights don't change them. Start from **Sprite Lit Shader Graph** for lit versions.

If something goes wrong

- **The dissolve slime still just vanishes:** the **On Died** entry has no object, or says **No Function**. Choose **DissolveOnDeath > Dissolve ()**.
- **The copy appears but doesn't dissolve:** a **Reference** in your graph isn't exactly `_DissolveAmount`. Check the spelling and the underscore, then **Save Asset**.
- **The copy is a white or pink square:** the **MainTex** property's Reference isn't `_MainTex`, or the graph wasn't saved.
- **It dissolves in big blobs, or all at once:** the material's **Noise** slot is empty, or **Noise Scale** is 0.
- **NullReferenceException in OutlineWhenNear:** nothing in the scene has the **Player** tag.
- **Every outline slime lights up at once:** check you used `.material`, not `.sharedMaterial`.

Try it

1. Rebuild the flash in Shader Graph: **Sample Texture 2D** (MainTex) into a **Lerp** (A = texture **RGBA**, B = a **Flash Color** property `_FlashColor`, T = a **Flash Amount** slider `_FlashAmount`) into **Base**

Color, texture **A** into **Alpha**. Make a material from it and put it on the Tint slime: PulseTint works unchanged.

2. A dissolve that goes the other way, so a slime *appears* when it spawns: count `_DissolveAmount` down from 1 to 0.
3. A red outline on enemies and gold on pickups: duplicate **Sprite Outline** and change **Outline Color**.
4. Harder: build the outline in Shader Graph. Sample the texture four times with the UV nudged up, down, left and right by one texel (**Tiling And Offset**, with the size from **Texel Size**), add the four alphas, **Saturate**, and multiply by **One Minus** the centre alpha. That's the outline mask: **Lerp** the colour to an **Outline Color** by it and use the **Maximum** of the two alphas.